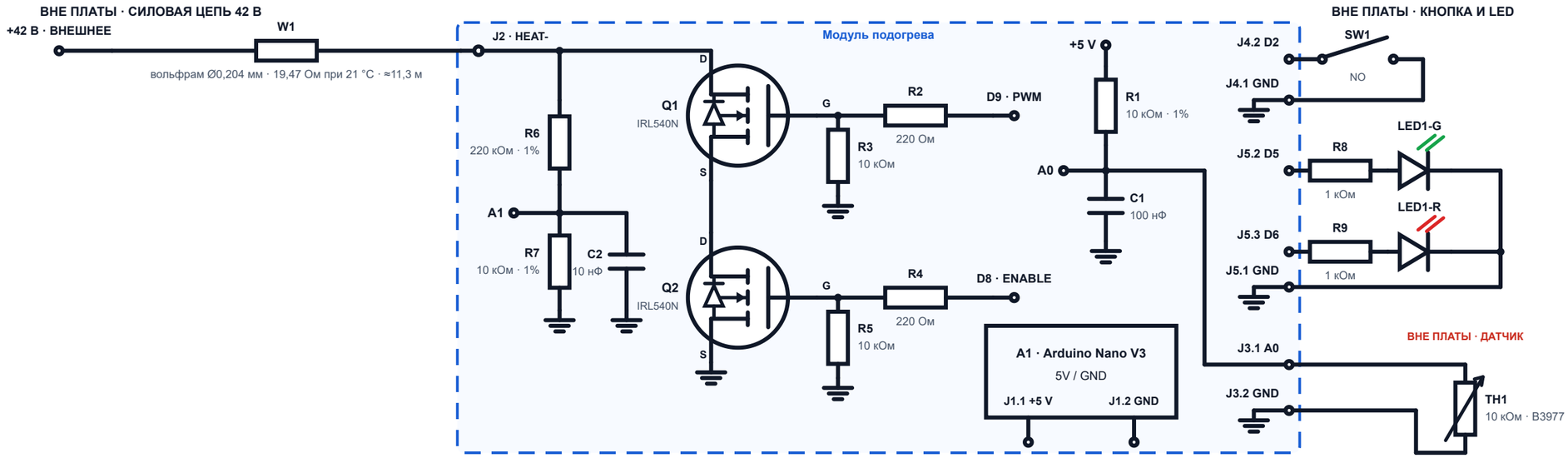
Подогрев руля 42 В - схема и алгоритм

Версия 1.3 · SW-H42-PB-07 · Arduino Nano V3 · измерено 70 Вт после прогрева · лист 1

Принципиальная схема



Пунктирная синяя рамка - граница макетной платы. Всё вне рамки - внешняя проводка и элементы.

Алгоритм

1. D8 и D9 переводятся в LOW до инициализации Arduino. Watchdog примерно на 1 с включается до EEPROM и самотеста.
2. EEPROM хранит OFF, ON или FAULT с инверсией и маркером записи. Повреждённая запись и распознанные watchdog/brownout без Power-on приводят к FAULT.
3. Восстановить ON/OFF из EEPROM, в том числе при Reset и неизвестном перезапуске. OFF не включает BOOST. Сохранённый FAULT имеет приоритет и остаётся заблокированным.
4. NTC: первый отсчёт АЦП отбрасывается, следующие 16 усредняются. Обрыв, КЗ и недопустимая температура дают FAULT. При 49 °C ключи закрываются немедленно после измерения.
5. Самотест: оба ключа OFF - A1 высокий; только Q2 ON - A1 высокий; только Q1 ON - A1 высокий; оба ON - A1 низкий. Полный тест обязателен до первого рабочего PWM; при тёплом старте он отложен до остывания.
6. Кнопка D2: антидребезг 35 мс; вне FAULT одно переключение ON/OFF на нажатие. В FAULT короткое нажатие игнорируется. Зажатую при запуске или до ошибки кнопку сначала отпустить.
7. При ON измерить температуру и проверить силовую цепь. На тёплом руле сохранить ON без тока, ждать остывания и полного теста. OFF кнопкой закрывает ключи и сохраняется в EEPROM.
8. Ниже 36,4 °C запускается BOOST: PWM=255. При достижении 36,4 °C (NTC около 6,11 кОм) либо через 5 минут включается HOLD.
9. HOLD: цель 33,2 °C (NTC около 7,00 кОм), PI-регулятор с защитой от накопления интеграла; PWM не выше 180. После BOOST нагрев ждёт остывания до цели.
10. При текущей или фильтрованной температуре от 39,9 °C оба ключа OFF; диагностические импульсы также запрещены. При 44 °C дополнительно фиксируется блокировка до 42 °C; кнопка её не сбрасывает.
11. Каждые 250 мс: NTC, затем силовая проверка при допустимой температуре. Пока полный тест не пройден, рабочий PWM запрещён. Диагностика завершается с ключами OFF.
12. Зелёный - OFF; красный - запрос нагрева; мигание 4 Гц - FAULT. В FAULT удерживать кнопку 5 с: проверка NTC, температуры ниже 39,9 °C и силовой цепи. Успех -> OFF; затем отпустить и нажать для ON. Неуспех -> FAULT; новая попытка только новым удержанием.

Выводы и индикация

Nano	Цепь	Назначение
D9	R2 -> Gate Q1	PWM
D8	R4 -> Gate Q2	ENABLE
A1	R6 / R7	самотест Q1, Q2 и цепи нагрева
A0	R1 / NTC	температура
D2	кнопка -> GND	ON/OFF; в FAULT удержание 5 с
D5/D6	R8/R9 -> LED	зелёный READY; красный HEAT/FAULT

Запуск и сброс ошибки

FAULT снимается кнопкой 5 с после успешного самотеста. Отключение питания и Reset ошибку не снимают. На всё время удержания сохраняется красное мигание; после успешной проверки - зелёный и OFF. Одна попытка на удержание.

Без сохранённого FAULT восстанавливается ON/OFF; на тёплом руле ON ждёт остывания. При утраченной причине сброса ON может восстановиться и после watchdog. Распознанные watchdog/brownout дают FAULT. Опция HEATER_OPTIBOOT_RESET_CAUSE=1 - только для проверенного Optiboot >=4.6.

Для самотеста нужны +5 В и 42 В. Если 42 В исчезнут раньше 5 В и Nano успеет проверить A1, сохранится FAULT. До эксплуатации проверить реальные фронты обоих питаний при запуске и остановке двигателя.

Все изменяемые электрические, температурные и NTC-параметры собраны в начале скетча. SUPPLY_V автоматически пересчитывает пороги A1; ток аппаратно не измеряется.

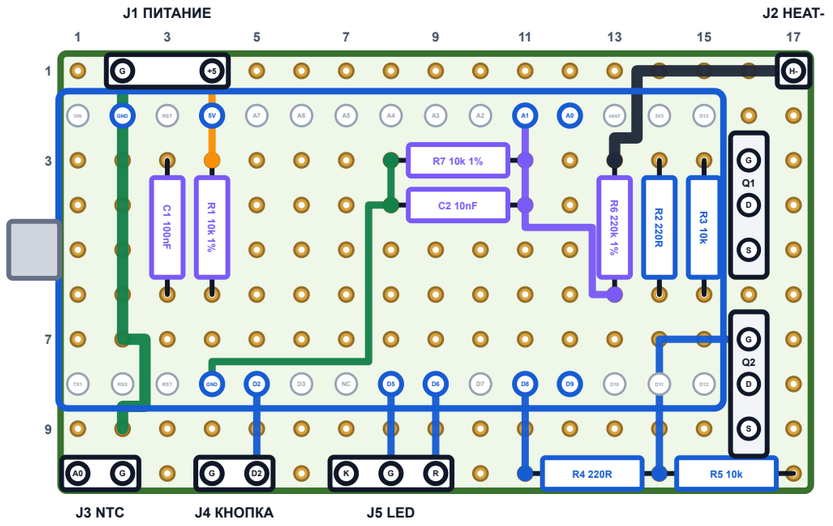
Загрузчик должен корректно обрабатывать watchdog: при цикле сбросов до скетча кнопка не работает. Нужен совместимый загрузчик или запуск через ISP без него.

A1 различает уровни напряжения, но не измеряет ток. Самотест и Q2 не заменяют независимую термозащиту и не защищают от одновременного пробоя обоих ключей.

Монтажная плата 17 x 10 отверстий

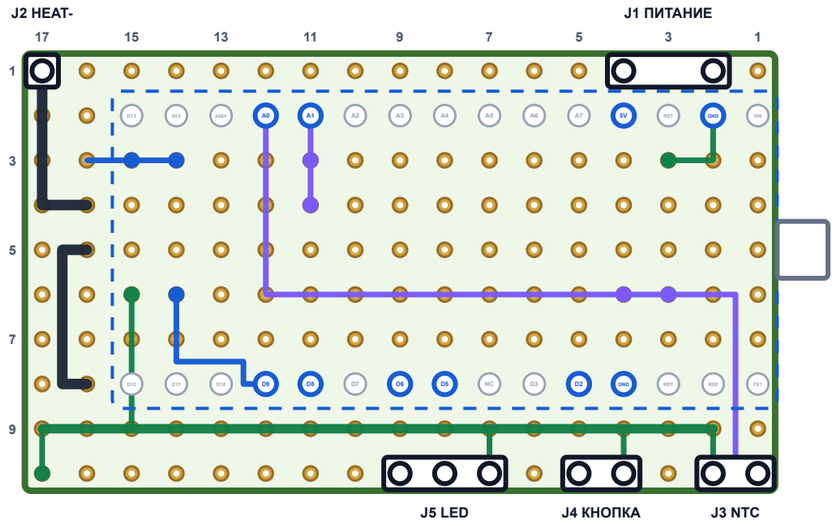
версия 1.3 · шаг 2,54 мм · сторона деталей и зеркальная сторона пайки

СТОРОНА ДЕТАЛЕЙ



17 x 10 отверстий · шаг 2,54 мм · плата ~40,6 x 22,9 мм

СТОРОНА ПАЙКИ - ЗЕРКАЛЬНО



17 x 10 отверстий · шаг 2,54 мм · плата ~40,6 x 22,9 мм

Цвет и сечение

- HEAT- / Q1 / Q2 : 0,5-0,75 мм²
- +5 В : 0,2-0,35 мм²
- Gate и GPIO : 0,2-0,35 мм²
- A0/A1 : 0,2-0,35 мм², вдали от PWM
- GND : силовая часть 0,5-0,75 мм²

+42 В и W1 находятся вне платы.

Все линии сплошные; сторона указана над видом платы.

Перемычки сверху выполняются изолированным проводом.

J1/J2 - питание и нагреватель; J3-J5 - сигнальные цепи.

J1: шаг 5,08 мм; J2: одиночная силовая клемма.

Контрольные точки

J1 +5	4,9-5,1 В
J1 GND	0 В; общая земля
J2 HEAT-	около 42 В при Q1 OFF; около 0 В при обоих ON
A0	около 2,5 В при 25 °C
A1	около 1,83 В при Q1 OFF
Gate Q1	D9 PWM 0-5 В; R3 -> GND
Gate Q2	0 В OFF; 5 В при разрешённом нагреве

Спецификация

Версия 1.3 · SW-H42-PB-07 · номиналы и внешние соединения · лист 3

Компоненты платы

Поз.	Кол.	Компонент	Назначение
A1	1	Arduino Nano V3, ATmega328P, 5 В	измеряет температуру, выполняет самотест и управляет нагревом
Q1,Q2	2	IRL540N, TO-220, N-MOSFET, 100 В	Q1 регулирует мощность PWM; Q2 аварийно разрывает силовую цепь
R1,R7	2	10 кОм, 1%	R1 задаёт измерительный делитель NTC; R7 ограничивает уровень сигнала A1
R2,R4	2	220 Ом	ограничивают импульсный ток затворов Q1 и Q2
R3,R5	2	10 кОм	гарантированно закрывают Q1 и Q2 при Reset или обрыве управления
R6	1	220 кОм, 1%, 0,25 Вт, рабочее напряжение не менее 100 В	вместе с R7 снижает напряжение силовой цепи до безопасного уровня A1
C1	1	100 нФ	фильтрует помехи в измерении температуры
C2	1	10 нФ	подавляет PWM-помехи в цепи самотеста A1

Внешние элементы

Поз.	Кол.	Компонент и параметры	Назначение
W1	1	вольфрам Ø0,204 мм; 19,47 Ом при 21 °C, четырёхпроводное измерение; расчётная длина около 11,3 м	единая нагревательная обмотка по всему ободу; при 42 В измерено около 70 Вт и 1,67 А после прогрева
TH1	1	Vishay NTCLE100E3103JB0, 10 кОм, B=3977 K	измеряет температуру обода для регулирования и защиты
SW1	1	кнопка без фиксации, контакт NO	ON/OFF коротким нажатием; сброс FAULT удержанием 5 с после самотеста
LED1	1	двухцветный LED, общий катод	показывает READY, HEAT и FAULT
R8,R9	2	1 кОм	ограничивают ток зелёного и красного кристаллов LED

Разъёмы

Разъём	Контакты
J1 ПИТАНИЕ	1: +5 В; 2: GND. Шаг 5,08 мм. На виде со стороны деталей слева GND, справа +5 В.
J2 HEAT-	1 силовая клемма: возврат от конца W1 к Drain Q1
J3 NTC	1: A0; 2: GND
J4 КНОПКА	1: GND; 2: D2
J5 LED	1: общий катод; 2: зелёный анод; 3: красный анод

Скетч Arduino Nano V3 - версия 1.3

steering_wheel_heater_v1_3.ino · строки 1-196 · копируемый текст с комментариями

```
/*
  Модуль подогрева 1.3. Arduino Nano V3 / ATmega328P, 5 В, 16 МГц.
  D9 -> R2 -> Q1 Gate; D8 -> R4 -> Q2 Gate. +42 V -> W1 -> Q1 -> Q2 -> GND.
  A1: R6 220k / R7 10k, C2 10nF. A0: +5 -> R1 10k -> NTC 10k B3977 -> GND,
  C1 100nF между A0 и GND. D2: кнопка на GND. D5/D6: зелёный/красный LED.

  FAULT снимается удержанием кнопки D2 5 с и успешным самотестом -> OFF.
  Reset и отключение питания не снимают сохранённый FAULT.
  ON/OFF восстанавливается из EEPROM, включая неизвестный перезапуск.
  Тёплый руль сохраняет ON, но ждёт остывания и полного силового самотеста.
  Распознанные watchdog/brownout -> FAULT. Причина - из MCUSR;
  для проверенного Optiboot >=4.6 с передачей MCUSR в r2 есть опция ниже.
  Загрузчик должен корректно обрабатывать watchdog: кнопка не лечит boot-loop.
  Самотест не является измерителем тока или независимой термозащитой.
*/
#include <Arduino.h>
#include <EEPROM.h>
#include <math.h>
#include <avr/wdt.h>

#define HEATER_VERSION "1.3"
#ifndef HEATER_OPTIBOOT_RESET_CAUSE
// Оставить 0; ставить 1 только для проверенного Optiboot >=4.6, передающего причину сброса через r2.
#define HEATER_OPTIBOOT_RESET_CAUSE 0
#endif
#if !defined(__AVR_ATmega328P__) || F_CPU != 16000000UL
#error "This firmware requires ATmega328P at 16 MHz"
#endif

uint8_t resetCause __attribute__((section(".noinit")));
#if HEATER_OPTIBOOT_RESET_CAUSE
uint8_t bootResetCause __attribute__((section(".noinit")));
void captureBootReset() __attribute__((used, naked, section(".init0")));
void captureBootReset() {
  __asm__ __volatile__("sts %0, r2" : "=m" (bootResetCause));
}
#endif
void earlySafetyInit() __attribute__((used, naked, section(".init3")));
void earlySafetyInit() {
  resetCause = MCUSR;
#if HEATER_OPTIBOOT_RESET_CAUSE
  if (resetCause == 0) resetCause = bootResetCause;
#endif
  MCUSR = 0;
  wdt_disable();
  // D8/PB0 и D9/PB1 в LOW до инициализации Arduino и EEPROM.
  PORTB &= static_cast<uint8_t>(~(_BV(PB0) | _BV(PB1)));
  DDRB |= _BV(PB0) | _BV(PB1);
}

// ================= НАСТРОЙКИ ПОЛЬЗОВАТЕЛЯ =================
// Для своей обмотки указать напряжение и холодное сопротивление, затем температуры BOOST/HOLD.
// Дроби писать через точку: 33.2f. После изменений заново скомпилировать и загрузить скетч.
// Электрические данные: расчёт I=U/R, P=U*U/R. Реальный ток и мощность модуль не измеряет.
constexpr float SUPPLY_V = 42.0f;             // В: фактическое питание обмотки; меняет расчёт I/P и пороги A1.
// SUPPLY_V не регулирует преобразователь: его выходное напряжение выставить вручную.
constexpr float HEATER_R_REF_OHM = 19.47f;    // Ом: измеренное холодное сопротивление; меняет расчёт I/P.
constexpr float HEATER_R_REF_C = 21.0f;       // °C: температура измерения сопротивления; только справочная запись.
constexpr float WIRE_DIAMETER_MM = 0.204f;    // мм: диаметр вольфрама; только справочная запись, на PWM не влияет.
constexpr float DESIGN_CURRENT_LIMIT_A = 3.0f; // A: допустимый расчётный ток аппаратуры, проверяется при компиляции.
constexpr float DESIGN_POWER_LIMIT_W = 120.0f;// Вт: допустимая расчётная мощность, проверяется при компиляции.
// Эти два предела не ограничивают реальный ток; задавать по возможностям собранной силовой цепи.

// Температуры относятся к месту установки NTC. После BOOST нагрев ждёт остывания до цели HOLD.
constexpr float BOOST_EXIT_C = 36.4f;         // °C: конец быстрого прогрева; для нашего NTC около 6,11 кОм.
constexpr float TARGET_C = 33.2f;             // °C: цель поддержания; выше значение - теплее, сейчас около 7,00 кОм.
constexpr float FORCE_OFF_C = 39.9f;          // °C: закрыть оба ключа, запретить и диагностические импульсы.
constexpr float SOFT_OFF_C = 44.0f;           // °C: включить дополнительную температурную блокировку.
constexpr float SOFT_RESET_C = 42.0f;         // °C: снять эту блокировку после остывания; рабочая цель отдельная.
constexpr float EMERGENCY_C = 49.0f;          // °C: записать FAULT; сброс кнопкой 5 с после успешного самотеста.
// Порядок: TARGET_C < BOOST_EXIT_C < FORCE_OFF_C < SOFT_OFF_C < EMERGENCY_C;
// SOFT_RESET_C < SOFT_OFF_C. Защитные температуры при повторении схемы оставить прежними.
constexpr uint32_t BOOST_MAX_MS = 300000;     // мс: предел времени BOOST; 300000 = 5 минут, затем HOLD.
constexpr uint8_t BOOST_PWM = 255;            // 0..255: мощность BOOST; 255 = 100%, 128 примерно 50%.
constexpr uint8_t HOLD_MAX_PWM = 180;         // 0..255: верхний предел мощности HOLD; 180 примерно 71%.
// PWM меняет среднюю мощность; ток во время включённого импульса определяется напряжением и обмоткой.
constexpr float KP = 35.0f;                   // PWM/°C: реакция на недогрев; больше - резче. Обычно не менять.
constexpr float KI = 0.8f;                    // PWM/(°C*c): устранение постоянного недогрева; обычно не менять.

// Менять следующие параметры при замене NTC, резисторов делителей или после калибровки.
constexpr float NTC_R25 = 10000.0f;           // Ом: сопротивление NTC при 25 °C, взять из его документации.
constexpr float NTC_FIXED = 10000.0f;         // Ом: установленный R1; используется для расчёта сопротивления NTC.
constexpr float NTC_BETA = 3977.0f;           // K: коэффициент B датчика; определяет пересчёт сопротивления в °C.
constexpr float NTC_OFFSET_C = 0.0f;          // °C: прибавка к показанию; +1.0f повышает его на 1 °C.
// Поправку NTC_OFFSET_C определять сравнением с термометром при установившейся температуре.
constexpr float ADC_REF_V = 5.0f;             // В: опорное напряжение АЦП, здесь питание Nano; влияет на пороги A1.
constexpr float A1_TOP_OHM = 220000.0f;       // Ом: установленный R6, верхнее плечо делителя самотеста.
constexpr float A1_BOTTOM_OHM = 10000.0f;     // Ом: установленный R7, нижнее плечо; оба номинала меняют пороги A1.
constexpr float A1_HIGH_RATIO = 0.35f;        // Доля расчётного A1: минимум для HIGH; обычно оставить 0.35f.
constexpr float A1_LOW_RATIO = 0.08f;         // Доля расчётного A1: максимум для LOW; обычно оставить 0.08f.
// Доли задают допуски самотеста, а не точность стабилизации питания. Пороги ниже вычисляются автоматически.
// ==========================================================

constexpr float NOMINAL_CURRENT_A = SUPPLY_V / HEATER_R_REF_OHM;
constexpr float NOMINAL_POWER_W = SUPPLY_V * NOMINAL_CURRENT_A;
constexpr float A1_EXPECTED_HIGH_ADC = SUPPLY_V * A1_BOTTOM_OHM /
    (A1_TOP_OHM + A1_BOTTOM_OHM) / ADC_REF_V * 1023.0f;
constexpr uint16_t DRAIN_HIGH_MIN =
    static_cast<uint16_t>(A1_EXPECTED_HIGH_ADC * A1_HIGH_RATIO + 0.5f);
constexpr uint16_t DRAIN_LOW_MAX =
    static_cast<uint16_t>(A1_EXPECTED_HIGH_ADC * A1_LOW_RATIO + 0.5f);

constexpr uint8_t PIN_PWM = 9, PIN_ENABLE = 8, PIN_BUTTON = 2;
constexpr uint8_t PIN_NTC = A0, PIN_DRAIN = A1, PIN_GREEN = 5, PIN_RED = 6;
constexpr uint32_t CONTROL_MS = 250, DEBOUNCE_MS = 35, BLINK_MS = 125;
constexpr uint32_t FAULT_RESET_MS = 5000;
constexpr uint16_t SETTLE_US = 600;

static_assert(SUPPLY_V > 0.0f && HEATER_R_REF_OHM > 0.0f,
              "Invalid heater electrical parameters");
static_assert(NOMINAL_CURRENT_A <= DESIGN_CURRENT_LIMIT_A &&
              NOMINAL_POWER_W <= DESIGN_POWER_LIMIT_W,
              "Calculated cold load exceeds configured hardware limits");
static_assert(TARGET_C < BOOST_EXIT_C && BOOST_EXIT_C < FORCE_OFF_C &&
              FORCE_OFF_C < SOFT_OFF_C && SOFT_RESET_C < SOFT_OFF_C &&
              SOFT_OFF_C < EMERGENCY_C, "Invalid temperature limits");
static_assert(A1_EXPECTED_HIGH_ADC < 1020.0f &&
              DRAIN_LOW_MAX < DRAIN_HIGH_MIN, "Invalid A1 divider or thresholds");

enum class Mode : uint8_t { Off, Boost, Hold, Fault };
enum class Fault : uint8_t {
  None, Sensor, Overheat, SupplyOrHeater, Q1, Q2, Conduction,
  Watchdog, Brownout, Stored, Eeprom
};
Mode mode = Mode::Off;
volatile Fault faultCode = Fault::None;  // Первый код до сброса FAULT или перезапуска; для отладки.
bool softCutoff = false;
bool powerStageReady = false;  // Полный самотест пройден; на тёплом старте отложен.
float temperatureC = NAN, filteredC = NAN, integral = 0.0f;
uint32_t boostStartedAt = 0, lastControlAt = 0, buttonChangedAt = 0;
uint32_t faultPressAt = 0;
bool buttonRaw = HIGH, buttonStable = HIGH, buttonArmed = false;
bool faultResetHeld = false;

// Формат 1.1: маркер, состояние, инверсия. Незавершённая запись -> FAULT.
constexpr int EE_MAGIC_ADDR = 0, EE_STATE_ADDR = 1, EE_INVERSE_ADDR = 2;
constexpr uint8_t EE_MAGIC = 0xB1, EE_OFF = 0x3C, EE_ON = 0xC3, EE_FAULT = 0x5A;
void tripFault(Fault reason);

bool heatingRequested() {
  return mode == Mode::Boost || mode == Mode::Hold;
}

void pwmWrite(uint8_t value) {
  analogWrite(PIN_PWM, value);
}

void gatesOff() {
  // Q2 закрывается первым: пробитый Q1 не мешает отключению нагрузки.
  digitalWrite(PIN_ENABLE, LOW);
  pwmWrite(0);
}

uint8_t loadState() {
  const uint8_t magic = EEPROM.read(EE_MAGIC_ADDR);
  const uint8_t state = EEPROM.read(EE_STATE_ADDR);
  const uint8_t inverse = EEPROM.read(EE_INVERSE_ADDR);
  // Чистая EEPROM и миграция старого формата всегда начинают с OFF.
  if ((magic == 0xFF && state == 0xFF && inverse == 0xFF) || magic == 0xA5)
    return EE_OFF;
  const bool validState = state == EE_OFF || state == EE_ON || state == EE_FAULT;
  if (magic != EE_MAGIC || !validState || inverse != static_cast<uint8_t>(~state))
    return 0;  // Повреждённая или оборванная запись; не восстанавливать ON.
  return state;
}

void saveState(uint8_t state) {
  if (EEPROM.read(EE_MAGIC_ADDR) == EE_MAGIC &&
      EEPROM.read(EE_STATE_ADDR) == state &&
      EEPROM.read(EE_INVERSE_ADDR) == static_cast<uint8_t>(~state)) return;
  EEPROM.update(EE_MAGIC_ADDR, 0);  // Сначала запретить чтение старой команды.
  EEPROM.update(EE_STATE_ADDR, state);
  EEPROM.update(EE_INVERSE_ADDR, static_cast<uint8_t>(~state));
  EEPROM.update(EE_MAGIC_ADDR, EE_MAGIC);  // Маркер действительности пишется последним.
  eeprom_busy_wait();  // Команда считается сохранённой только после записи.
}

void tripFault(Fault reason) {
  gatesOff();
  if (mode == Mode::Fault) return;  // Не перезаписывать EEPROM каждый цикл.
  mode = Mode::Fault;
  faultCode = reason;
  integral = 0.0f;
  softCutoff = false;
  powerStageReady = false;
  saveState(EE_FAULT);
}

uint16_t readAdc(uint8_t pin, uint8_t samples) {
  analogRead(pin);  // Первая конверсия после переключения мультиплексора отбрасывается.
  uint16_t sum = 0;
  for (uint8_t i = 0; i < samples; ++i) sum += static_cast<uint16_t>(analogRead(pin));
  return static_cast<uint16_t>((sum + samples / 2U) / samples);
}

bool sampleTemperature() {
```

Скетч Arduino Nano V3 - версия 1.3

steering_wheel_heater_v1_3.ino · строки 197-390 · копируемый текст с комментариями

```
  const uint16_t adc = readAdc(PIN_NTC, 16);
  if (adc <= 2 || adc >= 1021) { tripFault(Fault::Sensor); return false; }
  const float resistance = NTC_FIXED * adc / (1023.0f - adc);
  const float logRatio = static_cast<float>(logf(resistance / NTC_R25));
  const float inverseK = 1.0f / 298.15f + logRatio / NTC_BETA;
  const float value = 1.0f / inverseK - 273.15f + NTC_OFFSET_C;
  if (!isfinite(value) || value < -45.0f || value > 125.0f) {
    tripFault(Fault::Sensor); return false;
  }
  temperatureC = value;
  if (value >= EMERGENCY_C) { tripFault(Fault::Overheat); return false; }
  filteredC = isnan(filteredC) ? value : filteredC + 0.25f * (value - filteredC);
  if (value >= SOFT_OFF_C) softCutoff = true;
  else if (value <= SOFT_RESET_C) softCutoff = false;
  return true;
}

bool drainIsHigh() {
  delayMicroseconds(SETTLE_US);
  return readAdc(PIN_DRAIN, 4) >= DRAIN_HIGH_MIN;
}

bool checkPowerStage(bool testConduction) {
  // На входе и при любом выходе оба ключа закрыты; старый PWM не восстанавливается.
  gatesOff();
  if (!drainIsHigh()) { tripFault(Fault::SupplyOrHeater); return false; }
  digitalWrite(PIN_ENABLE, HIGH);  // Только Q2: исправный Q1 удерживает Drain высоким.
  const bool q1Ok = drainIsHigh();
  gatesOff();
  if (!q1Ok) { tripFault(Fault::Q1); return false; }
  delayMicroseconds(SETTLE_US);
  pwmWrite(255);  // Только Q1: исправный Q2 разрывает путь к GND.
  const bool q2Ok = drainIsHigh();
  gatesOff();
  if (!q2Ok) { tripFault(Fault::Q2); return false; }
  if (testConduction) {
    digitalWrite(PIN_ENABLE, HIGH);
    pwmWrite(255);  // Короткий импульс проверяет замыкание пути, но не измеряет ток.
    delayMicroseconds(SETTLE_US);
    const uint16_t adc = readAdc(PIN_DRAIN, 4);
    gatesOff();
    if (adc > DRAIN_LOW_MAX) { tripFault(Fault::Conduction); return false; }
    powerStageReady = true;
  }
  return true;
}

bool mustStopHeat() {
  // Текущая температура также ограничивает выход: фильтр не задерживает отключение.
  return softCutoff || temperatureC >= FORCE_OFF_C || filteredC >= FORCE_OFF_C;
}

void startHeating() {
  integral = 0.0f;
  boostStartedAt = millis();
  mode = temperatureC < BOOST_EXIT_C && filteredC < BOOST_EXIT_C ? Mode::Boost : Mode::Hold;
}

void stopHeating() {
  gatesOff();
  mode = Mode::Off;
  integral = 0.0f;
  // Гистерезис температурной блокировки не сбрасывается кнопкой.
}

uint8_t holdPwm(float dtSeconds) {
  // Использовать более горячее из текущего и фильтрованного значений: после
  // завершения BOOST не продолжать нагрев из-за запаздывания фильтра.
  const float controlC = max(temperatureC, filteredC);
  const float error = TARGET_C - controlC;
  const float proposed = constrain(integral + KI * error * dtSeconds, 0.0f,
                                   static_cast<float>(HOLD_MAX_PWM));
  const float output = KP * error + proposed;
  // Anti-windup: не накапливать интеграл дальше предела доступной мощности.
  if (!((output > HOLD_MAX_PWM && error > 0.0f) || (output < 0.0f && error < 0.0f)))
    integral = proposed;
  const float limited = constrain(KP * error + integral, 0.0f,
                                  static_cast<float>(HOLD_MAX_PWM));
  return static_cast<uint8_t>(limited + 0.5f);
}

void controlStep(uint32_t now, float dtSeconds) {
  if (mode == Mode::Fault) { gatesOff(); return; }
  if (!sampleTemperature()) return;
  if (mustStopHeat()) {
    gatesOff();  // До любых тестовых переключений; никаких импульсов выше порога.
    integral = 0.0f;
    if (mode == Mode::Boost) mode = Mode::Hold;
    return;
  }
  if (!checkPowerStage(!powerStageReady)) return;
  if (!heatingRequested()) return;
  if (mode == Mode::Boost && (temperatureC >= BOOST_EXIT_C ||
      filteredC >= BOOST_EXIT_C || static_cast<uint32_t>(now - boostStartedAt) >= BOOST_MAX_MS)) {
    mode = Mode::Hold;
    integral = 0.0f;
  }
  const uint8_t pwm = mode == Mode::Boost ? BOOST_PWM : holdPwm(dtSeconds);
  if (pwm != 0) { digitalWrite(PIN_ENABLE, HIGH); pwmWrite(pwm); }
}

void onButtonPress() {
  if (mode == Mode::Fault) return;
  if (heatingRequested()) { stopHeating(); saveState(EE_OFF); return; }
  gatesOff();
  if (!sampleTemperature()) return;  // Не использовать устаревший отсчёт.
  if (mustStopHeat()) powerStageReady = false;  // Принять ON без тока, проверить после остывания.
  else if (!checkPowerStage(true)) return;
  saveState(EE_ON);  // Запись завершена до разрешения рабочего нагрева.
  startHeating();
}

void tryClearFault() {
  gatesOff();
  filteredC = NAN;  // После ошибки оценивать свежую температуру, не старое значение фильтра.
  if (!sampleTemperature() || mustStopHeat()) return;
  if (!checkPowerStage(true)) return;  // Неисправность осталась: сохранить FAULT.
  saveState(EE_OFF);  // Сначала завершить запись; ни одного автоматического ON.
  stopHeating();
  faultCode = Fault::None;
}

void serviceButton(uint32_t now) {
  const bool raw = digitalRead(PIN_BUTTON) != LOW;
  if (raw != buttonRaw) { buttonRaw = raw; buttonChangedAt = now; }
  if (static_cast<uint32_t>(now - buttonChangedAt) < DEBOUNCE_MS) return;
  if (buttonStable == buttonRaw) {
    if (buttonStable == HIGH) buttonArmed = true;
    if (faultResetHeld && static_cast<uint32_t>(now - faultPressAt) >= FAULT_RESET_MS) {
      faultResetHeld = false;  // Одна попытка на удержание, даже если самотест не пройден.
      tryClearFault();
    }
    return;
  }
  buttonStable = buttonRaw;
  if (buttonStable == HIGH) { buttonArmed = true; faultResetHeld = false; }
  else if (buttonArmed) {
    buttonArmed = false;
    faultResetHeld = mode == Mode::Fault;  // Начать удержание заново уже после ошибки.
    faultPressAt = now;
    if (!faultResetHeld) onButtonPress();
  }
}

void updateLed(uint32_t now) {
  digitalWrite(PIN_GREEN, mode == Mode::Off ? HIGH : LOW);
  const bool red = mode == Mode::Fault ? (now / BLINK_MS) % 2U != 0 : heatingRequested();
  digitalWrite(PIN_RED, red ? HIGH : LOW);
}

void setup() {
  digitalWrite(PIN_PWM, LOW); pinMode(PIN_PWM, OUTPUT);
  digitalWrite(PIN_ENABLE, LOW); pinMode(PIN_ENABLE, OUTPUT);
  gatesOff();
  pinMode(PIN_BUTTON, INPUT_PULLUP);
  pinMode(PIN_GREEN, OUTPUT); pinMode(PIN_RED, OUTPUT);
  digitalWrite(PIN_GREEN, LOW); digitalWrite(PIN_RED, LOW);
  analogReference(DEFAULT);
  wdt_enable(WDTO_1S);  // Защищает и стартовые проверки, не только loop().
  buttonRaw = digitalRead(PIN_BUTTON) != LOW;
  buttonStable = buttonRaw;
  buttonChangedAt = millis();
  const uint8_t saved = loadState();
  const uint8_t known = _BV(PORF) | _BV(EXTRF) | _BV(BORF) | _BV(WDRF);
  const bool knownReset = resetCause != 0 && (resetCause & static_cast<uint8_t>(~known)) == 0;
  const bool powerOn = knownReset && (resetCause & _BV(PORF)) != 0 &&
                       (resetCause & (_BV(EXTRF) | _BV(WDRF))) == 0;
  if (knownReset && (resetCause & _BV(WDRF)) != 0) tripFault(Fault::Watchdog);
  else if (knownReset && (resetCause & _BV(BORF)) != 0 && !powerOn) tripFault(Fault::Brownout);
  else if (saved == 0) tripFault(Fault::Eeprom);
  else if (saved == EE_FAULT) tripFault(Fault::Stored);
  else if (sampleTemperature()) {
    // Тёплый старт не стирает ON; полный тест обязателен перед первым рабочим PWM.
    if (mustStopHeat() || checkPowerStage(true)) {
      if (saved == EE_ON) startHeating();
      else { stopHeating(); saveState(EE_OFF); }
    }
  }
  lastControlAt = millis();
  updateLed(lastControlAt);
  wdt_reset();
}

void loop() {
  const uint32_t now = millis();
  const uint32_t elapsed = now - lastControlAt;  // Корректно при переполнении millis().
  if (elapsed >= CONTROL_MS) {
    lastControlAt = now;
    controlStep(now, min(elapsed, 1000UL) / 1000.0f);
  }
  serviceButton(millis());
  updateLed(millis());
  wdt_reset();  // Только после выполнения всего прохода; внутри ожиданий не кормить.
}
```